

BACKEND ENGINEERING DOCUMENTATION

House of Hackney

Node.js · Express.js · MongoDB — B2C & B2B Commerce Platform

Version 1.0.0 • August 2026

43

COLLECTIONS

156

ENDPOINTS

8

CORE WORKFLOWS

Table of Contents

1. Project Overview & Technology Stack
2. Complete Folder Structure
3. Architecture Pattern — Repository → Service → Controller
4. Code Style & Conventions
5. Environment Configuration
6. Middleware Stack
7. Shared Utilities & Base Classes
8. Database Models — Complete Schema Reference (43 Collections)
9. Entity Relationship Diagram
10. Complete API Route Map (156 Endpoints)
11. Core Business Workflow Diagrams
12. Security Architecture
13. Deployment & Startup

1. Project Overview & Technology Stack

House of Hackney is an enterprise-grade **B2C + B2B luxury e-commerce RESTful API** for a British interior design brand. It supports multi-tenant retail commerce, wholesale B2B trade accounts, RBAC administration, Cloudinary media management, and transactional email via SMTP.

Technology Stack

Layer	Technology	Version
Runtime	Node.js	v24.x
Framework	Express.js	v5.2.1
Database	MongoDB (Mongoose ODM)	v9.8.0
Authentication	JWT (Access + Refresh Token Pair)	jsonwebtoken v9.0.3
Password Hashing	bcrypt	v6.0.0
Input Validation	express-validator	v7.3.2
File Upload	Multer (memory storage) → Cloudinary CDN	multer v2.2.0
Media CDN	Cloudinary v2	v2.10.0
Image Processing	Sharp	v0.35.3
Email (SMTP)	Nodemailer + EJS Templates	v9.0.5
PDF Generation	PDFKit	v0.19.1
Security	Helmet (HTTP Headers)	v8.3.0
Logging	Morgan (HTTP Logger)	v1.11.0
Module System	ES Modules ("type": "module")	—

2. Complete Folder Structure

```
HOUSE/
├── SYSTEM_ARCHITECTURE_AND_MODELS_WORKFLOW.md
├── src/
│   ├── .env # Environment variables
│   ├── server.js # Entry point - starts Express + MongoDB
│   ├── app.js # Express app setup (routes, middleware)
│   ├── package.json # Dependencies & scripts
│   ├── seed.js # Database seeder script
│   ├── seed_images_catalog.js # Catalog image seeder
│   ├── upload_images_to_cloudinary.js # Bulk Cloudinary uploader
│   ├──
│   ├── config/
│   │   ├── Database.js # MongoDB connection + admin seed
│   │   ├── jwt.js # JWT configuration (secrets, expiry)
│   │   └── cloudinary.js # Cloudinary v2 SDK config
│   ├──
│   ├── middlewares/
│   │   ├── auth.middleware.js # JWT Bearer token verification
│   │   ├── authorize.middleware.js # Role-based access control (RBAC)
│   │   ├── permission.middleware.js # Granular permission check
│   │   ├── error.middleware.js # Global error handler
│   │   ├── validateRequest.js # express-validator result handler
│   │   ├── upload.middleware.js # Multer file upload config
│   │   ├── rateLimit.middleware.js # In-memory sliding window rate limiter
│   │   ├── optionalAuth.middleware.js # Optional auth (for public + auth routes)
│   │   ├── logger.middleware.js # Request logger
│   │   └── notFound.middleware.js # 404 handler
│   ├──
│   ├── routes/
│   │   └── index.js # Central route registry (/api/v1/*)
│   ├──
│   ├── shared/
│   │   ├── database/
│   │   │   ├── BaseRepository.js # Generic Mongoose CRUD operations
│   │   │   ├── BaseService.js # Business logic base class
│   │   │   └── BaseController.js # HTTP handler base class
│   │   ├── errors/
│   │   │   ├── AppError.js # Custom error class (message + statusCode)
│   │   │   └── ErrorCodes.js # Standardized error codes
│   │   ├── responses/
│   │   │   └── ApiResponse.js # Standardized JSON response wrapper
│   │   ├── constants/
│   │   │   ├── product.constants.js # PRODUCT_STATUS, PRODUCT_TYPES, etc.
│   │   │   ├── inventory.constants.js
│   │   │   ├── currency.constants.js
│   │   │   ├── mediaFolder.js
│   │   │   ├── seo.constants.js
│   │   │   └── ... (14 files)
│   │   ├── enums/
│   │   │   ├── user.enum.js # Enum value mappings
│   │   │   ├── order.enum.js
│   │   │   ├── product.enum.js
│   │   │   └── ... (7 files)
│   │   ├── cache/
│   │   │   └── # Cache utilities
│   │   ├── helpers/
│   │   │   └── # Helper functions
│   │   ├── pagination/
│   │   │   └── # Pagination utilities
│   │   ├── populate/
│   │   │   └── # Mongoose populate configs
│   │   ├── query/
│   │   │   └── # Query builder utilities
│   │   ├── regex/
│   │   │   └── # Regex pattern library
│   │   ├── validators/
│   │   │   └── # Shared validation helpers
│   │   ├── events/
│   │   │   └── # Event emitter system
│   │   ├── jobs/
│   │   │   └── # Background job handlers
│   │   └── logger/
│   │       └── # Logger utility
│   ├──
│   ├── templates/
│   │   └── email/
│   │       ├── otp.ejs # OTP verification email template
│   │       └── welcome.ejs # Welcome email template
│   ├──
│   ├── utils/
│   │   ├── uploadResponse.js # Standardized upload response
│   │   └── deleteCloudinaryFile.js # Cloudinary deletion utility
│   └──
```

```

├─ images/                                # Local image assets
├─ postman_collections/                   # 26 Postman collection JSON files
├─   house_master.postman_collection.json # Master (156 endpoints)
├─   house_auth.postman_collection.json
├─   house_product.postman_collection.json
├─   house_order.postman_collection.json
├─   house_trade.postman_collection.json
├─   ... (26 total files)
└─ modules/                              # ← Domain module root
    ├─ auth/                              # Authentication & RBAC
    │   ├─ index.js                       # Auth module barrel export
    │   ├─ auth/                          # Core Auth (Register, Login, OTP, JWT)
    │   ├─ role/                          # Role CRUD
    │   ├─ permission/                   # Permission CRUD
    │   ├─ module/                       # System Module CRUD
    │   ├─ otp/                          # OTP Generation & Verification
    │   ├─ session/                      # User Session Management
    │   └─ token/                         # JWT Token Service
    ├─ user/                              # User Management
    ├─ catalog/                           # Catalog Taxonomy (10 submodules)
    │   ├─ Category/ Brand/ Collection/ Colour/ Designer/
    │   └─ Material/ Pattern/ Theme/ Style/ Room/
    ├─ product/                           # Product Engine (11 submodules)
    │   ├─ product/ Variant/ Inventory/ Pricing/ product-media/
    │   └─ attribute/ specification/ Seo/ related/ review/ filter/
    ├─ commerce/                          # B2C Commerce (8 submodules)
    │   ├─ Wishlist/ Cart/ address/ Coupon/
    │   └─ Order/ Payment/ Shipment/ invoice/
    ├─ trade/                             # B2B Wholesale Trade (6 submodules)
    │   ├─ tradeProfile/ Company/ tradeTier/
    │   └─ tradePricing/ quotation/ creditLimit/
    ├─ media/                             # Cloudinary Media Library
    ├─ counter/                           # Auto-Increment Counter Utility
    └─ userToken/                         # User Token Store

```

Each Domain Module Contains (Standard File Layout)

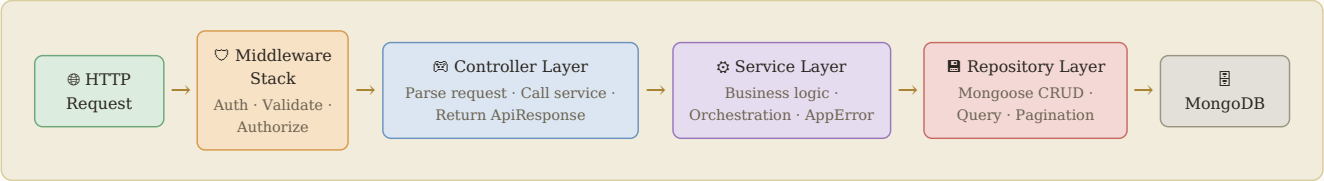
```

📁 module-name/
├─ module.model.js           # Mongoose Schema & Model
├─ module.repository.js      # Data access layer (extends BaseRepository)
├─ module.service.js         # Business logic layer (extends BaseService)
├─ module.controller.js      # HTTP handler layer (extends BaseController)
├─ module.routes.js          # Express Router definitions
├─ module.validation.js      # express-validator rules
└─ index.js                  # Barrel export

```

3. Architecture Pattern — Repository → Service → Controller

The project strictly follows a **3-Layer Architecture** (Clean Architecture inspired):



Layer Responsibilities

Layer	Base Class	Responsibility
Controller	BaseController	HTTP request/response handling. Delegates to Service. Returns ApiResponse .
Service	BaseService	Business rules, cross-module calls, data transformation. Throws AppError .
Repository	BaseRepository	Direct Mongoose operations — create, find, update, soft-delete, paginate.

Base Class Method Inventory

BaseRepository provides:

Method	Description
create(data)	Insert single document
bulkCreate(data[])	Insert multiple documents
findById(id)	Find by ObjectId
findOne(filter)	Find single matching document
findAll(filter)	Find all matching documents
findActive(filter)	Find where status=ACTIVE and deletedAt=null
update(id, data)	Update by ID with runValidators
updateOne(filter, data)	Update first matching document
delete(id)	Hard delete by ID
softDelete(id)	Set deletedAt timestamp
restore(id)	Clear deletedAt
exists(filter)	Check document existence
count(filter)	Count matching documents
bulkDelete(ids[])	Mass delete by IDs
paginate(filter, options)	Paginated query with sort, populate, projection

BaseService mirrors all BaseRepository methods, adding a layer of abstraction for business logic extension.

BaseController provides default CRUD handlers: create , getAll , getById , update , delete — each with try/catch and next(error) .

4. Code Style & Conventions

Language & Module System

- **ES Modules** (`import/export`) — configured via `"type": "module"` in `package.json`
- **No TypeScript** — pure JavaScript with JSDoc comments where needed

Naming Conventions

Element	Convention	Example
Files	<code>camelCase.type.js</code>	<code>product.model.js</code> , <code>auth.service.js</code>
Classes	<code>PascalCase</code>	<code>ProductService</code> , <code>BaseRepository</code>
Variables	<code>camelCase</code>	<code>orderNumber</code> , <code>shippingAddress</code>
Constants	<code>UPPER_SNAKE_CASE</code>	<code>PRODUCT_STATUS</code> , <code>JWT_ACCESS_SECRET</code>
Enums	<code>UPPER_SNAKE_CASE</code>	<code>ACTIVE</code> , <code>PENDING</code> , <code>APPROVED</code>
Routes	<code>kebab-case</code> URLs	<code>/api/v1/trade/profiles</code>
DB Collections	<code>lowercase plural</code>	<code>users</code> , <code>products</code> , <code>roles</code>
Mongoose Models	<code>PascalCase singular</code>	<code>User</code> , <code>Product</code> , <code>Role</code>

Controller Pattern

```
// Every controller method follows this exact pattern:
async methodName(req, res, next) {
  try {
    const result = await service.businessMethod(req.body);
    return new ApiResponse(res, 200, "Message.", result).send();
  } catch (error) {
    next(error); // → error.middleware.js
  }
}
```

API Response Format

Every API response follows this standardized JSON structure:

```
{
  "success": true,
  "message": "Product created successfully.",
  "data": { "...": "..." },
  "errors": null
}
```

Error Response Format

```
{
  "success": false,
  "message": "Invalid email or password.",
  "data": null,
  "errors": null
}
```

Validation Error Response (422)

```
{
  "success": false,
  "message": "Validation Failed",
  "errors": [
    {
      "type": "field",
      "msg": "Email is required",
      "path": "email",
      "location": "body"
    }
  ]
}
```

Export Pattern

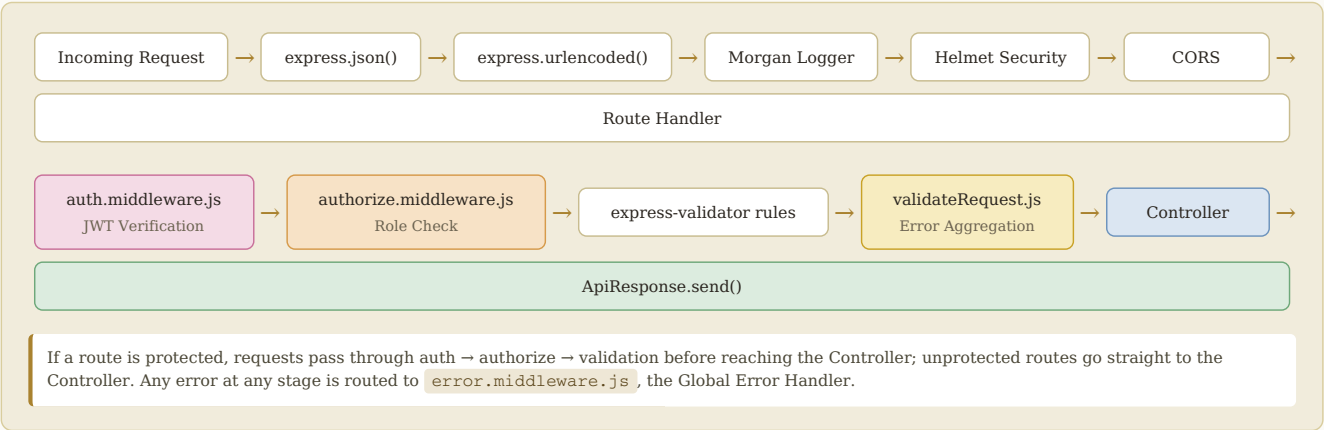
- All services, controllers, and repositories are exported as **singleton instances** (`export default new ClassName()`)
- Models are exported as Mongoose model instances (`export default mongoose.model("Name", schema)`)

5. Environment Configuration

Variable	Purpose	Example
NODE_ENV	Runtime environment	development
PORT	Server listen port	5000
MONGODB_URI	MongoDB connection string	mongodb://127.0.0.1:27017/house_of_hackney
JWT_ACCESS_SECRET	JWT access token secret	HouseOfHackney@Access#2026
JWT_REFRESH_SECRET	JWT refresh token secret	HouseOfHackney@Refresh#2026
JWT_ACCESS_EXPIRES_IN	Access token lifetime	15m
JWT_REFRESH_EXPIRES_IN	Refresh token lifetime	30d
SMTP_HOST	Email SMTP server	smtp.gmail.com
SMTP_PORT	SMTP port	587
SMTP_SECURE	Use TLS	false
SMTP_USER	SMTP username	email@gmail.com
SMTP_PASS	SMTP app password	xxxx xxxx xxxx xxxx
EMAIL_FROM	Sender display name	"House of Hackney <email>"
ADMIN_EMAIL	Default admin email	admin@example.com
ADMIN_PASSWORD	Default admin password	Admin@123
CLOUDINARY_CLOUD_NAME	Cloudinary cloud name	your_cloud_name
CLOUDINARY_API_KEY	Cloudinary API key	your_api_key
CLOUDINARY_API_SECRET	Cloudinary API secret	your_api_secret

6. Middleware Stack

Request processing pipeline (in execution order):



Middleware Details

Middleware	File	Purpose
Auth	auth.middleware.js	Extracts <code>Bearer</code> token from <code>Authorization</code> header → verifies JWT → attaches <code>req.user</code> , <code>req.token</code> , <code>req.jwtPayload</code>
Authorize	authorize.middleware.js	Checks if <code>req.user.roles[] .slug</code> includes any of the required role slugs (e.g., <code>["ADMIN", "SUPER_ADMIN"]</code>)
Permission	permission.middleware.js	Granular check: does <code>req.user.roles[] .permissions[] .permission.slug</code> match the required permission slug
Validate	validateRequest.js	Collects <code>express-validator</code> errors → returns <code>422</code> with error array
Upload	upload.middleware.js	Multer memory storage, allows JPEG/PNG/WebP/AVIF/SVG up to 10MB
Rate Limit	rateLimit.middleware.js	In-memory sliding window limiter (default: 10 req / 15 min per IP+URL)
Error	error.middleware.js	Catches all errors, returns <code>{ success: false, message }</code> with appropriate status code

7. Shared Utilities & Base Classes

AppError Class

```
class AppError extends Error {
  constructor(message, statusCode = 500, details = null) {
    super(message);
    this.statusCode = statusCode;
    this.details = details;
  }
}

// Usage: throw new AppError("Product not found.", 404);
```

ApiResponse Class

```
class ApiResponse {
  constructor(res, statusCode, message, data = null, errors = null) { /* ... */ }
  send() {
    return this.res.status(this.statusCode).json({
      success: this.statusCode >= 200 && this.statusCode < 300,
      message, data, errors
    });
  }
}

// Usage: return new ApiResponse(res, 201, "Created.", product).send();
```

Shared Constants

File	Exports
product.constants.js	PRODUCT_STATUS : DRAFT, READY, PUBLISHED, ARCHIVED
	PRODUCT_VISIBILITY : PUBLIC, PRIVATE, HIDDEN
	PRODUCT_TYPES : WALLPAPER, FABRIC, FURNITURE, LIGHTING, PAINT, ACCESSORY
inventory.constants.js	Stock status enums
currency.constants.js	Default currency configuration
mediaFolder.js	Cloudinary folder paths
seo.constants.js	SEO field constraints

8. Database Models — Complete Schema Reference (43 Collections)

Module 1 · Authentication, Security & RBAC (6 Models)

✦ User Model (users collection)

Field	Type	Required	Details
firstName	String	✓	Max 50 chars, trimmed
lastName	String	✓	Max 50 chars, trimmed
email	String	✓	Unique, lowercase
countryCode	String	—	Default +91
phone	String	—	Unique, sparse
avatar	Object	—	{ public_id, url }
dateOfBirth	Date	—	
gender	Enum	—	MALE , FEMALE , OTHER , PREFER_NOT_TO_SAY
password	String	—	bcrypt hashed (12 rounds), select: false
loginProvider	Enum	—	EMAIL , GOOGLE , APPLE , FACEBOOK
registrationSource	Enum	—	WEB , ADMIN , GOOGLE , APPLE , FACEBOOK , IMPORT
passwordChangedAt	Date	—	Auto-updated on password change
lastLoginAt	Date	—	
tokenVersion	Number	—	Incremented on password change to invalidate old JWTs
roles	[ObjectId]	—	→ Role collection
isEmailVerified	Boolean	—	Default false
isTradeAccount	Boolean	—	Default false
tradeProfile	ObjectId	—	→ TradeProfile
language	String	—	Default en
currency	String	—	Default INR
loginAttempts	Number	—	Account lock after 5 failed attempts
isLocked	Boolean	—	15-minute lock on brute force
status	Enum	—	ACTIVE , INACTIVE , BLOCKED , SUSPENDED , DELETED
accountType	Enum	—	PERSONAL , BUSINESS

Pre-save hooks: Auto-hash password with bcrypt (12 rounds), auto-increment tokenVersion. **Instance methods:** comparePassword(candidatePassword) , toJSON() (strips password). **Query helpers:** .active() — filters status=ACTIVE and deletedAt=null. **Virtual:** displayName getter → \${firstName} \${lastName}.

✦ Role Model (roles collection)

Field	Type	Details
name	String	Uppercase, required. E.g., ADMIN , USER
displayName	String	Human-readable name
slug	String	Unique, lowercase. Used for authorization checks

description	String	
permissions	Array	[[{ permission: → Permission, grantedAt, grantedBy: → User }]]
priority	Number	Lower = higher authority
level	Number	Hierarchy level
isDefault	Boolean	Auto-assigned to new users
isSystem	Boolean	Cannot be deleted
canLoginAdmin	Boolean	Admin panel access flag
canLoginWebsite	Boolean	Website access flag
status	Enum	ACTIVE , INACTIVE , ARCHIVED

✦ Permission Model

Field	Type	Details
name	String	E.g., PRODUCT_READ
slug	String	URL-safe identifier
module	ObjectId	→ Module
action	String	READ , WRITE , DELETE , etc.
status	Enum	ACTIVE , INACTIVE , ARCHIVED

✦ Module Model

Field	Type	Details
name	String	E.g., PRODUCTS , COMMERCE
slug	String	
description	String	
icon	String	Dashboard icon identifier
route	String	Admin panel route
parent	ObjectId	→ Module (self-referencing hierarchy)
status	Enum	ACTIVE , INACTIVE , ARCHIVED

✦ Session Model

Field	Type	Details
user	ObjectId	→ User
refreshTokenHash	String	Hashed refresh token for validation
deviceId	String	Client device fingerprint
deviceName	String	E.g., Chrome on Windows
ipAddress	String	Client IP
userAgent	String	Full user-agent string
expiresAt	Date	Session expiry time

✦ OTP Model

Field	Type	Details
email	String	Target email address
otpHash	String	Hashed 6-digit OTP
purpose	Enum	EMAIL_VERIFICATION , PASSWORD_RESET
expiresAt	Date	OTP expiry time
isUsed	Boolean	Prevents replay attacks

Module 2 • Catalog Taxonomy (10 Models)

All catalog models share a similar simple structure:

Field	Type	Present In
name	String (required)	All 10 models
slug	String (unique, lowercase)	All 10 models
description	String	All 10 models
image / banner	ObjectId → Media	Category, Brand
colorHex / code	String	Colour
parentCategory	ObjectId → Category	Category (self-referencing tree)
level	Number	Category (hierarchy depth)
isFeatured	Boolean	Category
showInMenu	Boolean	Category
website	String	Brand
status	Enum	All (ACTIVE , INACTIVE)

Models: Category , Brand , Collection , Colour , Designer , Material , Pattern , Theme , Style , Room

Module 3 • Product Engine (12 Models)

✦ Product Model (products collection) — Master Entity

Field	Type	Details
name	String	Required, max 250 chars
slug	String	Unique, lowercase
sku	String	Unique, uppercase
barcode	String	
productType	Enum	WALLPAPER , FABRIC , FURNITURE , LIGHTING , PAINT , ACCESSORY
shortDescription	String	Max 500 chars
description	String	Rich text description
category	ObjectId	→ Category (required)
brand	ObjectId	→ Brand
collection	ObjectId	→ Collection
designer	ObjectId	→ Designer
material	ObjectId	→ Material
colours	[ObjectId]	→ Colour []
patterns	[ObjectId]	→ Pattern []
themes	[ObjectId]	→ Theme []
styles	[ObjectId]	→ Style []
rooms	[ObjectId]	→ Room []
isFeatured	Boolean	Featured product flag
isBestSeller	Boolean	Best seller flag
isNewArrival	Boolean	New arrival flag
isTradeAvailable	Boolean	Available for B2B trade
isCustomizable	Boolean	
searchKeywords	[String]	Full-text search optimization
status	Enum	DRAFT , READY , PUBLISHED , ARCHIVED

visibility	Enum	PUBLIC , PRIVATE , HIDDEN
publishedAt	Date	Publication timestamp
publishedBy	ObjectId	→ User

Indexes: category , brand , collection , designer , material , status , visibility , isFeatured , isBestSeller , isNewArrival , createdAt

✦ Variant Model

Field	Type	Details
product	ObjectId	→ Product (required)
title	String	Variant name
slug	String	Unique
sku	String	Unique
barcode	String	
attributes	Array	[[attribute: String, value: String]]
status	Enum	ACTIVE , INACTIVE

✦ ProductPricing Model

Field	Type	Details
product	ObjectId	→ Product
currency	String	Default INR
basePrice	Number	Cost price
sellingPrice	Number	Retail price
compareAtPrice	Number	Strikethrough price
tradePrice	Number	Wholesale price
taxClass	String	Tax category

✦ ProductInventory Model

Field	Type	Details
product	ObjectId	→ Product
warehouse	String	Warehouse location code
totalStock	Number	
reservedStock	Number	
availableStock	Number	
trackInventory	Boolean	Enable stock tracking

✦ Attribute Model

Field	Type	Details
name	String	E.g., Material Composition
slug	String	
displayName	String	
type	Enum	TEXT , NUMBER , SELECT , MULTISELECT , BOOLEAN , COLOR , IMAGE
values	Array	[[label, slug, colorCode, image → Media]]
status	Enum	ACTIVE , INACTIVE

Other Product Models (Summary)

Model	Key Fields	Relations
ProductMedia	product , thumbnail , gallery []	→ Product, → Media
Specification	product , specifications[{ key, value }]	→ Product
Seo	product , metaTitle , metaDescription , canonicalUrl , keywords []	→ Product, → Media
Related	product , relatedProducts []	→ Product (self-referencing)
Review	product , user , rating (1-5), title , comment , status	→ Product, → User, → Variant, → Order, → Media
FilterGroup	name , code , type , options [] , displayOrder	None (standalone)

Module 4 · B2C Commerce (8 Models)

✦ Cart Model (carts collection)

Field	Type	Details
user	ObjectId	→ User (unique — one cart per user)
items	Array	[[{ product → Product, variant → Variant, quantity, unitPrice, discount, tax, total, addedAt }]]
subtotal	Number	Pre-discount total
discount	Number	Total discount amount
tax	Number	Total tax
shipping	Number	Shipping fee
grandTotal	Number	Final total
currency	String	Default INR
coupon	Object	{ code, discountType: PERCENTAGE/FIXED, discountValue }
status	Enum	ACTIVE , ABANDONED , CONVERTED
expiresAt	Date	Cart expiry

✦ Order Model (orders collection)

Field	Type	Details
orderNumber	String	Unique readable number (e.g., ORD-123456-7890)
user	ObjectId	→ User
items	Array	Snapshot: [{ product, variant, name, sku, unitPrice, quantity, discount, tax, total }]
shippingAddress	Object	Full address snapshot: { fullName, phone, email, country, state, city, postalCode, addressLine1, addressLine2, landmark }
billingAddress	Object	Same structure as shipping
subtotal	Number	
discount	Number	
tax	Number	
shippingFee	Number	
grandTotal	Number	

currency	String	
coupon	Object	{ code , discountType , discountValue , discountAmount }
paymentStatus	Enum	PENDING , PAID , FAILED , REFUNDED
paymentMethod	Enum	COD , CARD , UPI , NETBANKING , WALLET
orderStatus	Enum	PENDING , CONFIRMED , PROCESSING , SHIPPED , DELIVERED , CANCELLED , RETURNED
notes	String	Customer instructions
cancelledAt	Date	
cancellationReason	String	

Other Commerce Models (Summary)

Model	Key Fields	Relations
Wishlist	user , items[{ product , variant , addedAt }]	→ User, → Product, → Variant
Address	user , fullName , phone , addressLine1/2 , city , state , postalCode , country , type , isDefault	→ User
Coupon	code , discountType , discountValue , minOrderAmount , maxDiscountAmount , validFrom/Until , usageLimit , applicableTo	→ Category, → Product, → Brand, → User
Payment	order , user , gateway , paymentMethod , amount , transactionId , status , paidAt	→ Order, → User
Shipment	order , carrier , trackingNumber , trackingUrl , status , shippedAt , estimatedDelivery , deliveredAt	→ Order, → User
Invoice	invoiceNumber , order , payment , shipment , items[] , subtotal , taxAmount , totalAmount , status	→ Order, → Payment, → Shipment, → User

Module 5 · B2B Wholesale Trade (6 Models)

Model	Key Fields	Relations
TradeProfile	user , company , gstNumber , vatNumber , businessType , phone , status , tier , creditLimit , usedCredit , rejectionReason	→ User, → Company, → TradeTier
Company	name , country , taxNumber , registrationNumber , website , email , phone , logo , status	→ User
TradeTier	name (BRONZE/SILVER/GOLD/PLATINUM), discountPercentage , creditLimit , paymentTerm , status	→ User
TradePricing	product , variant , tradeTier , customPrice , discountPercentage , status	→ Product, → Variant, → TradeTier
Quotation	quoteNumber , user , company , items[{ product , variant , quantity , unitPrice , total }] , subtotal , discount , grandTotal , status , notes	→ User, → Company, → Product, → Order
CreditLimit	company , user , totalCredit , usedCredit , availableCredit , paymentTerm , status	→ Company, → User

System Utility Models

Model	Key Fields	Purpose
Media	filename , publicId , url , secureUrl , folder , mimeType , size , status	Cloudinary CDN asset registry
Counter	name , sequence	Auto-increment number generator (e.g., order numbers)
UserToken	user , token , type , expiresAt	Password reset / verification token store

9. Entity Relationship Diagram

Identity & Access

User → has many → **Role**
Role → contains → **Permission**
Permission → belongs to → **Module**
User → has → **Session / OTP**
User → has one → **TradeProfile**

Catalog & Product

Product → belongs to → **Category**
Product → has → **Brand / Collection / Designer / Material**
Product → many-to-many → **Colour / Pattern / Theme / Style / Room**
Product → 1:1 → **Pricing / Media / SEO / Specification**
Product → 1:many → **Variant / Review / Related**
Product → 1:1 → **Inventory**
ProductMedia → references → **Media**
Category → image/banner → **Media**

Commerce

User → has one → **Wishlist / Cart**
User → has many → **Address / Order**
Cart → contains items → **Product / Variant**
Order → ordered items → **Product**
Order → 1:1 → **Payment / Shipment / Invoice**
Coupon → applicable to → **Category / Product**

B2B Trade

TradeProfile → works for → **Company**
TradeProfile → assigned → **TradeTier**
TradePricing → custom price for → **Product** at → **TradeTier**
Quotation → requested by → **Company**, for → **Product[]**
CreditLimit → credit for → **Company**

10. Complete API Route Map (156 Endpoints)

Authentication & Security

Method	Endpoint	Description	Auth
POST	/api/v1/auth/register	Register new user account	✗
POST	/api/v1/auth/verify-email	Verify email OTP	✗
POST	/api/v1/auth/resend-otp	Resend OTP code	✗
POST	/api/v1/auth/login	User login	✗
POST	/api/v1/auth/refresh-token	Refresh JWT tokens	✗
GET	/api/v1/auth/me	Get current user profile	✓
POST	/api/v1/auth/logout	Logout user	✓
GET	/api/v1/auth/sessions	Get active sessions	✓
POST	/api/v1/auth/sessions/revoke	Revoke specific session	✓
POST	/api/v1/auth/forgot-password	Initiate password reset	✗
POST	/api/v1/auth/verify-reset-otp	Verify reset OTP	✗
POST	/api/v1/auth/reset-password	Reset password	✗
PATCH	/api/v1/auth/change-password	Change password	✓

Users & RBAC

Method	Endpoint	Description	Auth
GET	/api/v1/users	List all users	✓
GET	/api/v1/users/:id	Get user details	✓
PUT	/api/v1/users/:id	Update user profile	✓
DELETE	/api/v1/users/:id	Delete user	✓
GET/POST/PATCH/DELETE	/api/v1/roles/*	Role CRUD	✓ Admin
GET/POST/PATCH/DELETE	/api/v1/permissions/*	Permission CRUD	✓ Admin
GET/POST/PATCH/DELETE	/api/v1/modules/*	System Module CRUD	✓ Admin

Catalog Management

Method	Endpoint	Description	Auth
GET/POST/PUT/DELETE	/api/v1/categories/*	Category CRUD	✓ Admin for write
GET/POST/PUT/DELETE	/api/v1/brands/*	Brand CRUD	✓ Admin for write

Product Engine

Method	Endpoint	Description	Auth
GET/POST/PUT/DELETE	/api/v1/products/*	Product CRUD	✓ Admin for write
GET/POST/PUT/DELETE	/api/v1/variants/*	Variant CRUD	✓
GET/POST/PUT/DELETE	/api/v1/inventories/*	Inventory Management	✓
GET/POST/PUT/DELETE	/api/v1/attributes/*	Attribute CRUD	✓
GET/POST/PUT/DELETE	/api/v1/products/:productId/pricing	Pricing per product	✓
GET/POST/PUT/DELETE	/api/v1/products/:productId/media	Media per product	✓
GET/POST/PUT/DELETE	/api/v1/products/:productId/specifications	Specs per product	✓
GET/POST/PUT/DELETE	/api/v1/products/:productId/seo	SEO per product	✓

GET/POST/DELETE	/api/v1/products/:productId/related	Related products	✓
GET/POST	/api/v1/products/:productId/reviews	Reviews per product	✓
GET/POST/PUT/DELETE	/api/v1/filters/*	Filter Group CRUD + Faceted Aggregation	Varies

B2C Commerce

Method	Endpoint	Description	Auth
GET/POST/DELETE	/api/v1/wishlist/*	Wishlist management	✓
GET/POST/PATCH/DELETE	/api/v1/cart/*	Cart management	✓
GET/POST/PUT/PATCH/DELETE	/api/v1/addresses/*	Address CRUD	✓
GET/POST/PUT/DELETE	/api/v1/coupons/*	Coupon CRUD	✓ Admin for write
GET/POST/PATCH	/api/v1/orders/*	Order lifecycle	✓
GET/POST	/api/v1/payments/*	Payment processing	✓
GET/POST/PATCH	/api/v1/shipments/*	Shipment management	✓
GET/POST/PATCH	/api/v1/invoices/*	Invoice generation	✓

B2B Wholesale Trade

Method	Endpoint	Description	Auth
GET/POST/PATCH	/api/v1/trade/profiles/*	Trade profile management	✓
GET/POST/PUT/DELETE	/api/v1/trade/companies/*	Company CRUD	✓
GET/POST/PUT	/api/v1/trade/tiers/*	Trade tier management	✓ Admin
GET/POST	/api/v1/trade/pricing/*	Custom trade pricing	✓ Admin
GET/POST/PATCH	/api/v1/trade/quotations/*	Quotation workflow	✓
GET/POST	/api/v1/trade/credits/*	Credit limit management	✓ Admin

System & Media

Method	Endpoint	Description	Auth
GET/POST/DELETE	/api/v1/media/*	Media library CRUD	✓
POST	/api/v1/counter	Auto-increment counter	✓

11. Core Business Workflow Diagrams

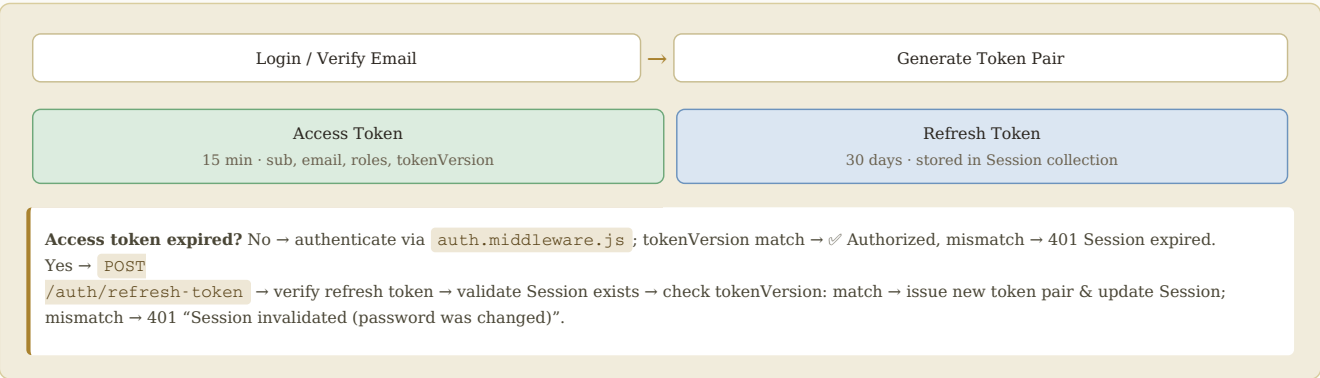
Workflow 1 — User Registration & Email Verification

Customer	→ API	POST /auth/register {email, password, firstName, lastName}
API	→ User Collection	Create User (isEmailVerified = false)
API	→ OTP Collection	Generate 6-digit OTP, hash with bcrypt, save
API	→ Gmail SMTP	Send OTP email (otp.ejs template)
SMTP	→ Customer	📧 "Your OTP: 123456"
API	→ Customer	201 Created (verification pending)
Customer	→ API	POST /auth/verify-email {email, otp}
API	→ OTP Collection	Validate OTP hash & check expiry
API	→ User Collection	Set isEmailVerified = true, emailVerifiedAt = now
API	→ API	Generate Access + Refresh JWT Token Pair
API	→ Gmail SMTP	Send Welcome Email (welcome.ejs)
API	→ Customer	200 OK {accessToken, refreshToken, user}

Workflow 2 — Login with Account Lock Protection

Customer	→ Auth Service	POST /auth/login {email, password}
API	→ User Collection	findByEmail(email) with +password select
alt User Not Found → 401 "Invalid email or password"		
alt Account Locked (5 failed attempts) → 423 "Account locked for 15 minutes"		
alt Invalid Password → increment loginAttempts; if ≥ 5, set isLocked = true, lockUntil = now + 15min → 423; else → 401		
alt Email Not Verified → resend verification OTP → 403 "Email not verified. OTP sent."		
API	→ User Collection	Reset loginAttempts = 0, update lastLoginAt
API	→ API	Generate JWT Token Pair (sub, email, roles, tokenVersion)
API	→ Session Collection	Create Session (refreshToken, deviceId, ipAddress, ...)
API	→ Customer	200 OK {user, accessToken, refreshToken}

Workflow 3 — JWT Token Lifecycle



Workflow 4 — Order Lifecycle (Cart → Order → Payment → Shipment → Invoice)

Buyer	→ Cart Service	POST /cart/items {productId, variantId, quantity}
Cart	→ Cart	Calculate unitPrice × quantity, tax, discount, grandTotal
Cart	→ Buyer	<i>Cart updated (5 items, ₹12,500)</i>
Buyer	→ Address Service	POST /addresses {fullName, address, city, postalCode}
Addr	→ Buyer	<i>Address created</i>
Buyer	→ Order Service	POST /orders {shippingAddressId, paymentMethod}
Order	→ Cart	Fetch active cart with populated items
Order	→ Address	Fetch & snapshot shipping / billing address
Order	→ Order	Generate ORD-XXXXXX-YYYY order number
Order	→ Inventory	Deduct stock for each item (stock -= quantity)
Order	→ Order	Create Order document, status = CONFIRMED
Order	→ Cart	Clear cart (status → CONVERTED)
Order	→ Buyer	<i>Order created (ORD-123456-7890)</i>
Buyer	→ Payment Service	POST /payments {orderId, gateway: STRIPE, amount}
Pay	→ Order	Update paymentStatus → PAID, orderStatus → PROCESSING
Pay	→ Buyer	<i>Payment successful</i>
ADMIN DISPATCHES ORDER		
Shipment	→ Order	POST /shipments {orderId, carrier, trackingNumber} → orderStatus → SHIPPED
Invoice	→ Invoice	POST /invoices {orderId} → generate INV-XXXX tax invoice
Shipment	→ Buyer	<i>Parcel delivered → orderStatus → DELIVERED</i>

Workflow 5 — Order Cancellation & Stock Reversal

Buyer	→ Order Service	PATCH /orders/:id/cancel {reason}
Order	→ Order	Validate status ≠ SHIPPED / DELIVERED / CANCELLED
LOOP — FOR EACH ORDER ITEM		
Order	→ Inventory	Find inventory by product → restock: stock += item.quantity
Order	→ Order	Set orderStatus = CANCELLED, cancelledAt = now
Order	→ Buyer	<i>Order cancelled, stock restored</i>

Workflow 6 — B2B Trade Account & Quotation Flow

STEP 1 — COMPANY REGISTRATION

Designer → **Trade Service** POST /trade/companies {name, country, taxNumber}

Trade → **Designer** Company created

STEP 2 — TRADE ACCOUNT APPLICATION

Designer → **Trade Service** POST /trade/profiles/apply {phone, businessType, gstNumber}

Trade → **Designer** Application submitted (status: PENDING)

STEP 3 — ADMIN REVIEWS APPLICATION

Admin → **Trade Service** GET /trade/profiles/pending

Admin → **Trade Service** PATCH /trade/profiles/:id/review {status: APPROVED, tierId: GOLD}

Trade → **Trade** Assign GOLD tier (25% discount, NET_30 payment)

Trade → **Designer** ✓ Trade account APPROVED

STEP 4 — BULK QUOTATION REQUEST

Designer → **Quotation Service** POST /trade/quotations/request {items: [{product, qty: 50}]}

Quote → **Designer** Quotation QUOTE-2026-001 (status: REQUESTED)

STEP 5 — NEGOTIATE & APPROVE

Admin → **Quotation Service** PATCH /trade/quotations/:id/negotiate {discount: 15%}

Quote → **Designer** Updated pricing (NEGOTIATING)

Designer → **Quotation Service** PATCH /trade/quotations/:id/approve

Quote → **Designer** Quotation APPROVED

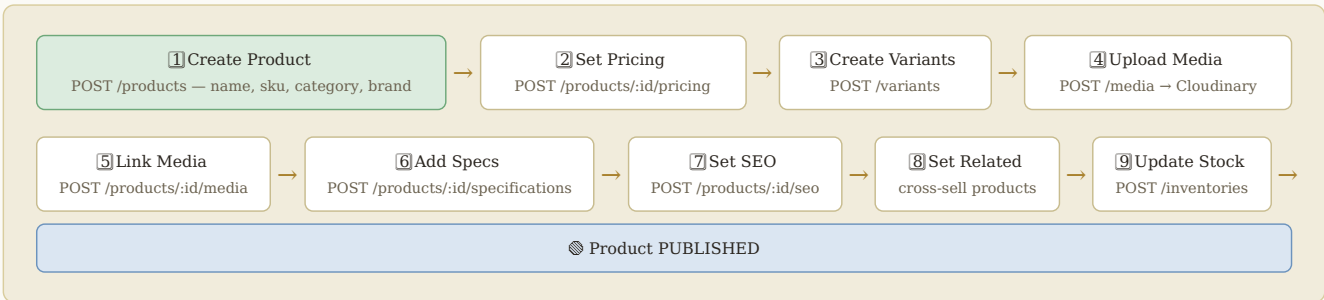
STEP 6 — CONVERT TO ORDER

Designer → **Quotation Service** POST /trade/quotations/:id/convert {shippingAddressId}

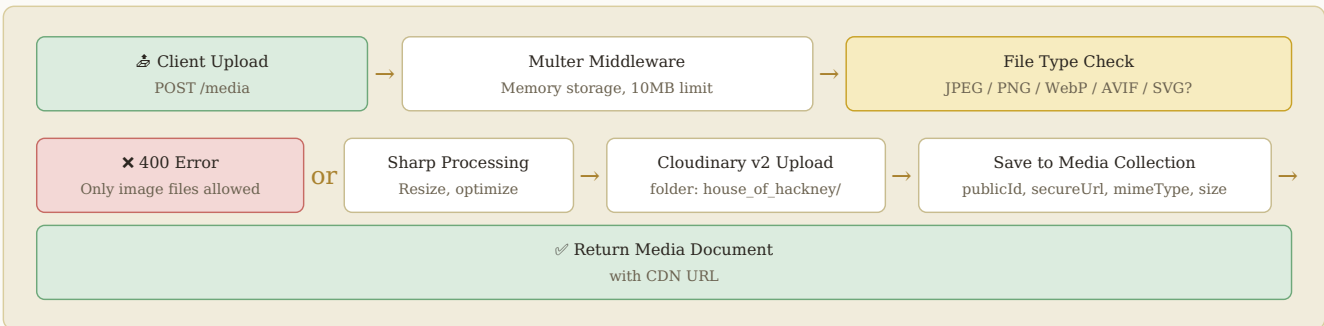
Quote → **Quote** Create B2B Order with NET_30 credit terms

Quote → **Designer** Order placed with trade pricing

Workflow 7 — Product Creation with Full Enrichment



Workflow 8 — Cloudinary Media Upload Pipeline

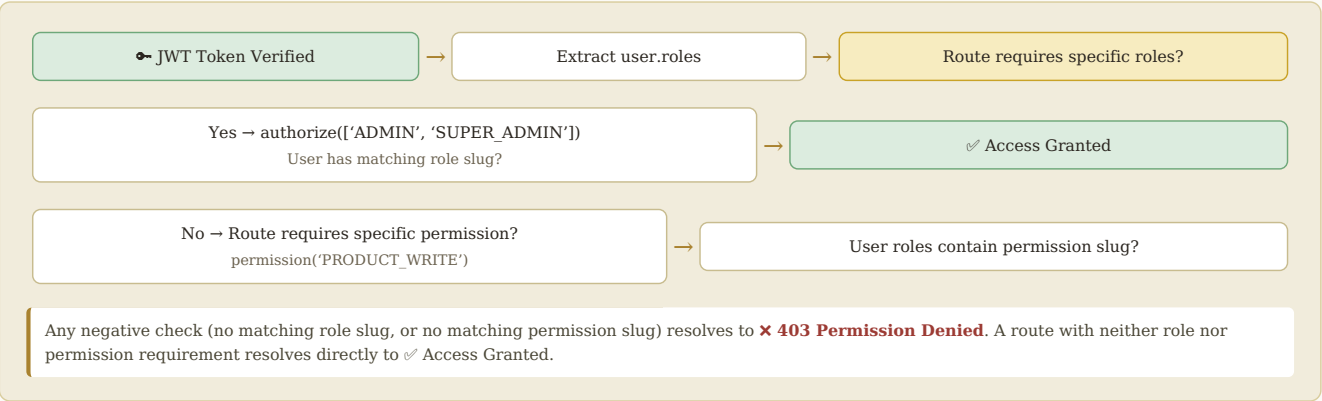


12. Security Architecture

Authentication Security Features

Feature	Implementation
Password Hashing	bcrypt with 12 salt rounds
JWT Dual Token	Short-lived access (15m) + long-lived refresh (30d)
Token Versioning	<code>tokenVersion</code> field — password change invalidates ALL existing tokens
Session Management	Each device has a separate session record; individual or bulk revocation
Account Locking	5 failed login attempts → 15-minute lock (<code>isLocked</code> , <code>lockUntil</code>)
OTP Verification	6-digit hash-based OTP with expiry for email verification and password reset
HTTP Security Headers	Helmet middleware for XSS, HSTS, CSP, etc.
Rate Limiting	In-memory sliding window (10 req / 15 min per IP+URL)
Input Validation	express-validator on every POST/PUT/PATCH route
Soft Delete	<code>deletedAt</code> timestamp pattern — data is never permanently lost
Audit Trails	<code>createdBy</code> , <code>updatedBy</code> , <code>createdAt</code> , <code>updatedAt</code> on all documents
CORS	Configurable origin whitelist

RBAC Access Control Flow



13. Deployment & Startup

NPM Scripts

Command	Purpose
npm run dev	Start with nodemon (hot-reload)
npm start	Production start
npm run seed	Seed database with sample data
npm run seed:images	Seed catalog images
npm run upload:cloudinary	Bulk upload local images to Cloudinary

Startup Sequence

